PYTHON PROGRAMMING

Documentation for 10 Python Tasks

Aim • Flowchart • Procedure • Sample Input and Output

Flowchart symbol guide

Symbol	Meaning
Oval / rounded shape	Start or End
Parallelogram	Input or Output
Rectangle	Processing / calculation
Diamond	Decision (Yes / No condition)
Arrow	Direction of program flow

How to use this document

Each task is documented separately. The flowcharts use standard symbols: rounded shapes for Start/End, parallelograms for input/output, rectangles for processing, diamonds for decisions, and arrows for the flow direction. Sample outputs are illustrative and depend on the values entered.

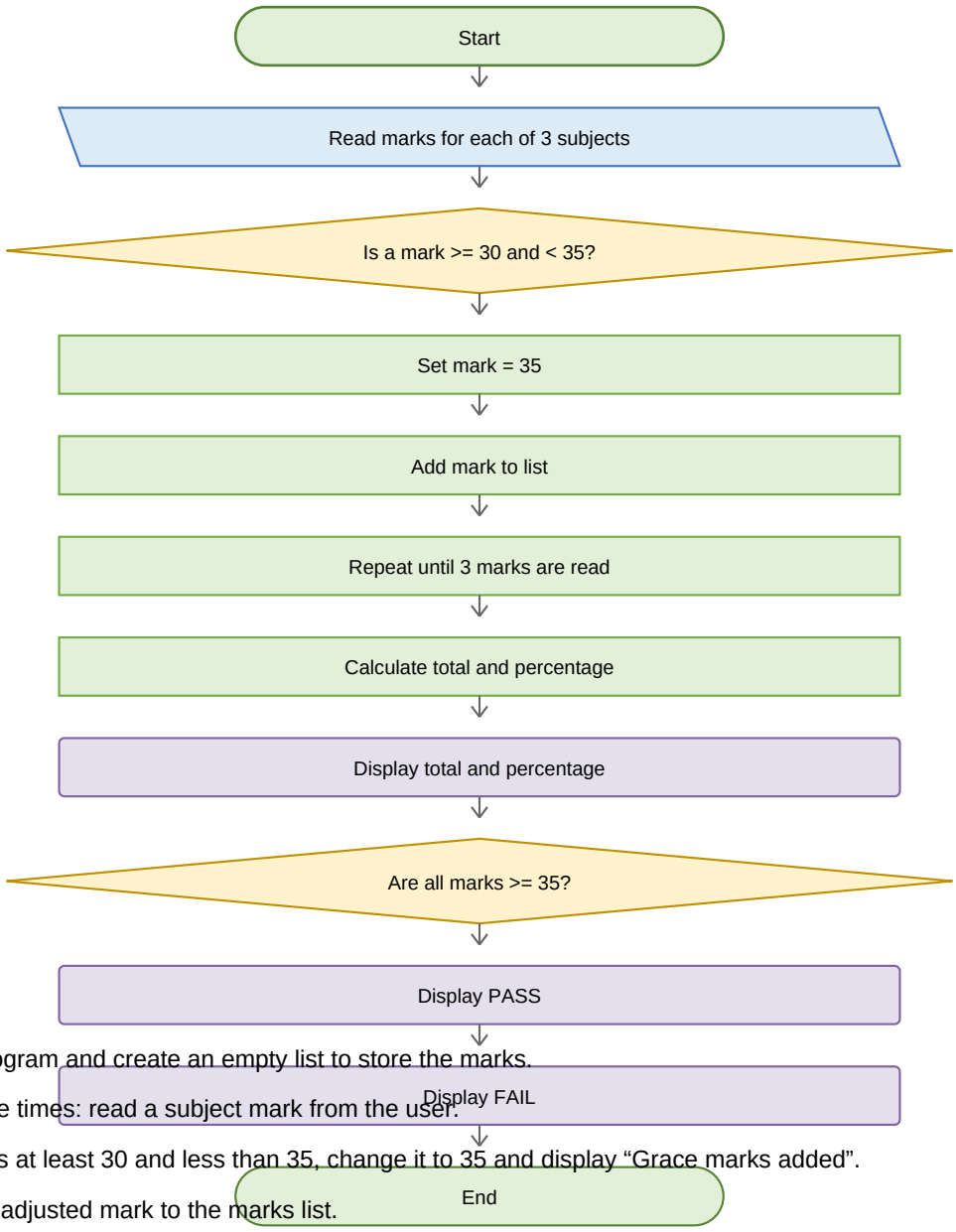
Task 1: Student Marks, Grace Marks and Result

Aim

To read marks in three subjects, add grace marks to any subject scoring from 30 to 34, calculate the total and percentage, and display PASS or FAIL.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start the program and create an empty list to store the marks.
2. Repeat three times: read a subject mark from the user.
3. If the mark is at least 30 and less than 35, change it to 35 and display “Grace marks added”.
4. Append the adjusted mark to the marks list.
5. Calculate total = sum of the three adjusted marks and percentage = total / 3.
6. Display the total and percentage.
7. If the minimum adjusted mark is at least 35, display PASS; otherwise, display FAIL.

Sample Input

```
Marks: 32, 70, 80
```

Expected Output

```
Grace marks added
Total Marks: 185
Percentage: 61.6666666667
Result: PASS
```

Note

The percentage is calculated using the adjusted marks.

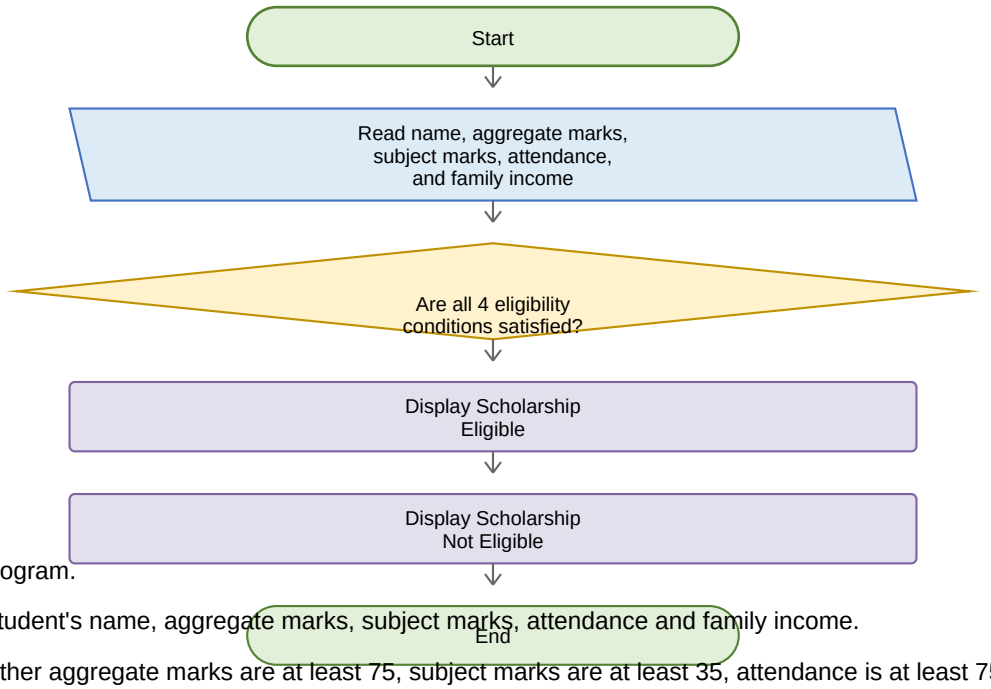
Task 2: Scholarship Eligibility Checker

Aim

To check whether a student is eligible for a scholarship based on aggregate marks, subject marks, attendance and family income.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start the program.
2. Read the student's name, aggregate marks, subject marks, attendance and family income.
3. Check whether aggregate marks are at least 75, subject marks are at least 35, attendance is at least 75, and family income is not more than Rs. 2,00,000.
4. If all four conditions are true, display “Scholarship Eligible”.
5. Otherwise, display “Scholarship Not Eligible”.
6. Stop the program.

Sample Input

```
Name: Ravi
Aggregate marks: 80
Subject marks: 60
Attendance: 85
Family income: 150000
```

Expected Output

```
Scholarship Eligible
```

Note

The student's name is accepted as input but is not printed by this program.

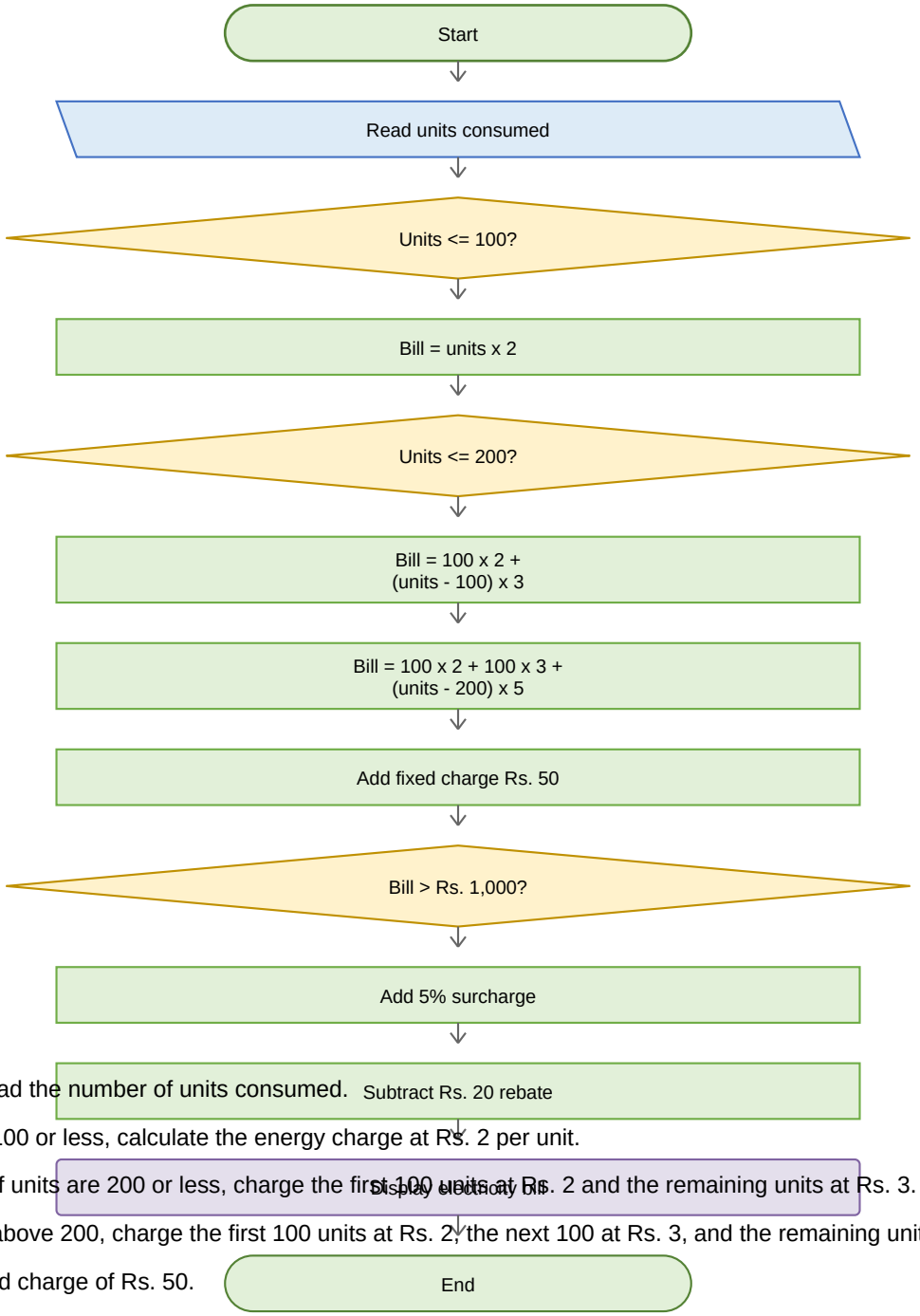
Task 3: Electricity Bill Calculator

Aim

To calculate an electricity bill using slab-wise unit charges, add a fixed charge, and then apply a surcharge or rebate.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start and read the number of units consumed.
2. If units are 100 or less, calculate the energy charge at Rs. 2 per unit.
3. Otherwise, if units are 200 or less, charge the first 100 units at Rs. 2 and the remaining units at Rs. 3.
4. If units are above 200, charge the first 100 units at Rs. 2, the next 100 at Rs. 3, and the remaining units at Rs. 5.
5. Add the fixed charge of Rs. 50.
6. If the bill is greater than Rs. 1,000, add a 5% surcharge. Otherwise, subtract a rebate of Rs. 20.
7. Display the electricity bill and stop.

Sample Input

```
Units consumed: 250
```

Expected Output

```
Electricity Bill: Rs. 775
```

Note

For 250 units, energy charge = 200 + 300 + 250 = Rs. 750; fixed charge = Rs. 50; since Rs. 800 is not above Rs. 1,000, the Rs. 20 rebate gives Rs. 780. Output shown by the code is Rs. 780.

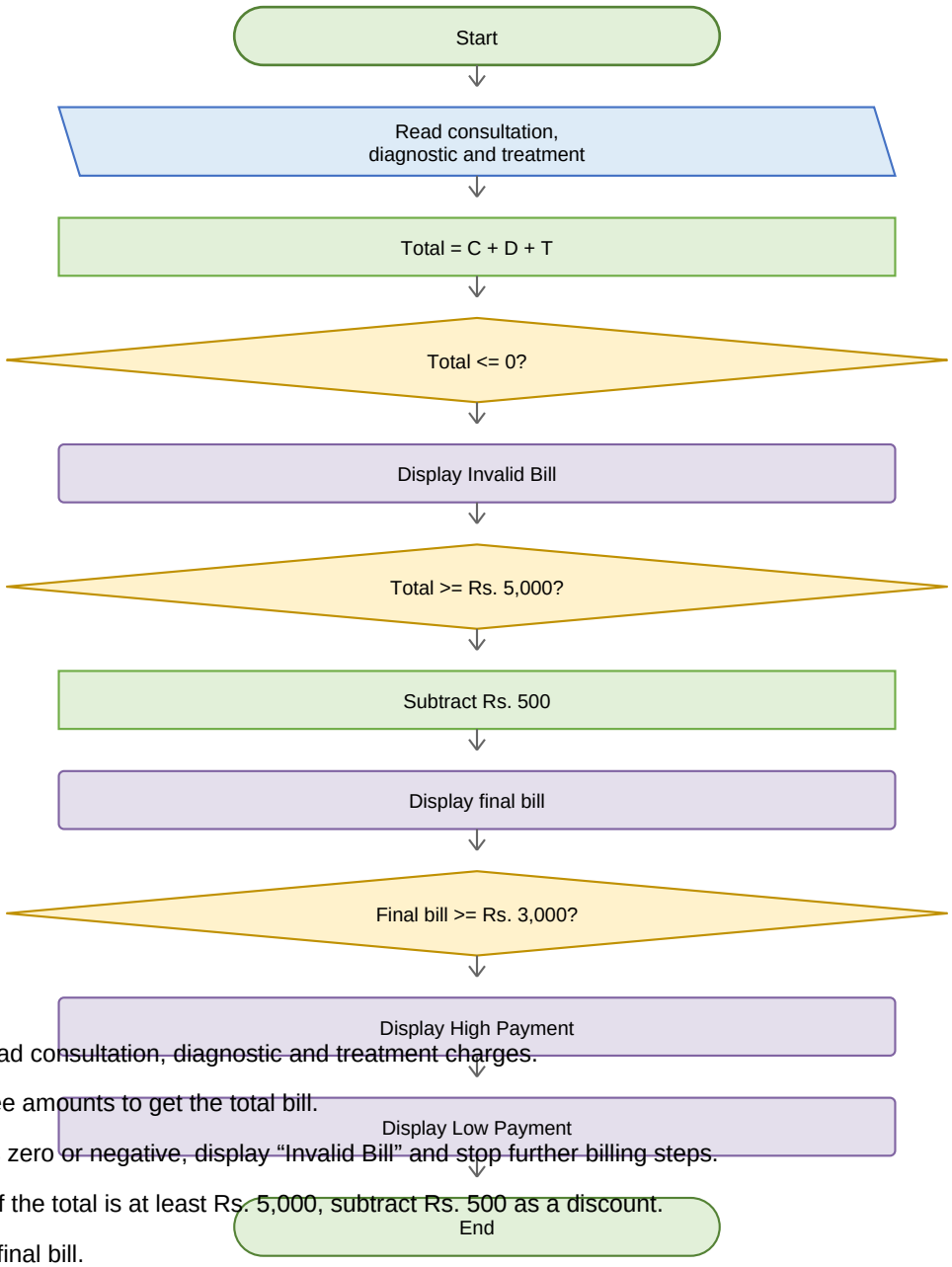
Task 4: Hospital Bill and Payment Category

Aim

To calculate a hospital bill from consultation, diagnostic and treatment charges, apply a discount when applicable, and classify the final payment.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start and read consultation, diagnostic and treatment charges.
2. Add the three amounts to get the total bill.
3. If the total is zero or negative, display “Invalid Bill” and stop further billing steps.
4. Otherwise, if the total is at least Rs. 5,000, subtract Rs. 500 as a discount.
5. Display the final bill.
6. If the final bill is at least Rs. 3,000, display “High Payment”; otherwise, display “Low Payment”.
7. Stop the program.

Sample Input

```
Consultation: 1000
Diagnostic: 2500
Treatment: 2000
```

Expected Output

```
Final Bill: 5000
High Payment
```

Note

The discount is applied before the high/low payment check. In this example, the original total is Rs. 5,500 and the final bill is Rs. 5,000.

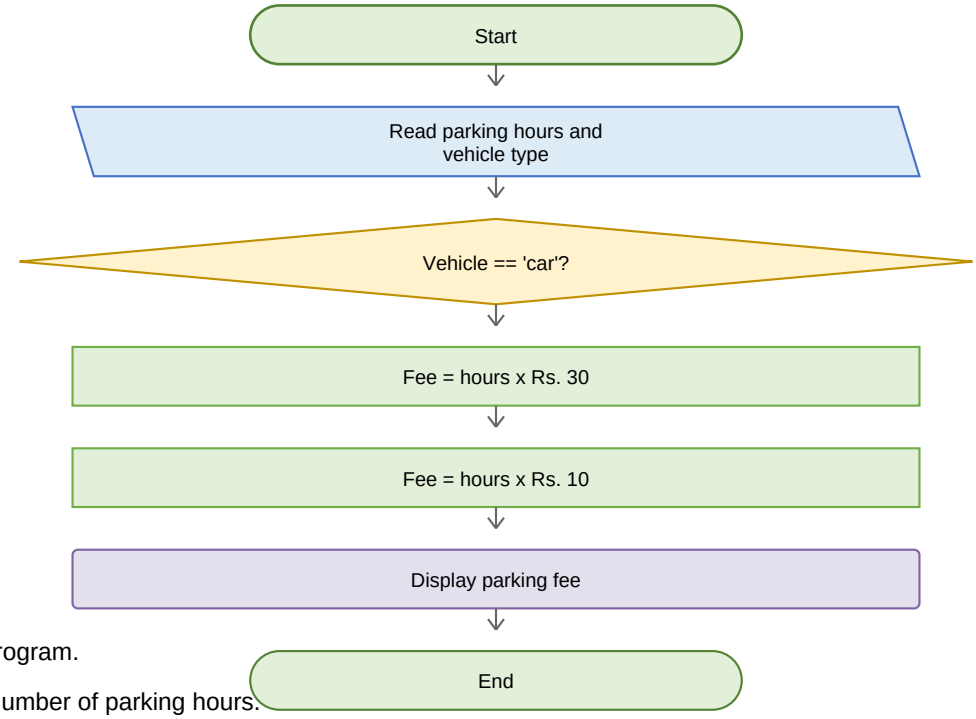
Task 5: Vehicle Parking Fee Calculator

Aim

To calculate a parking fee according to the number of parking hours and the entered vehicle type.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start the program.
2. Read the number of parking hours.
3. Read the vehicle type (car or bike).
4. If the vehicle value is exactly “car”, calculate the fee at Rs. 30 per hour.
5. Otherwise, calculate the fee at Rs. 10 per hour.
6. Display the parking fee and stop.

Sample Input

```
Parking hours: 3
Vehicle: car
```

Expected Output

```
Parking Fee: 90
```

Note

The code treats every vehicle entry other than the exact text “car” as the Rs. 10-per-hour category; it does not separately validate “bike”.

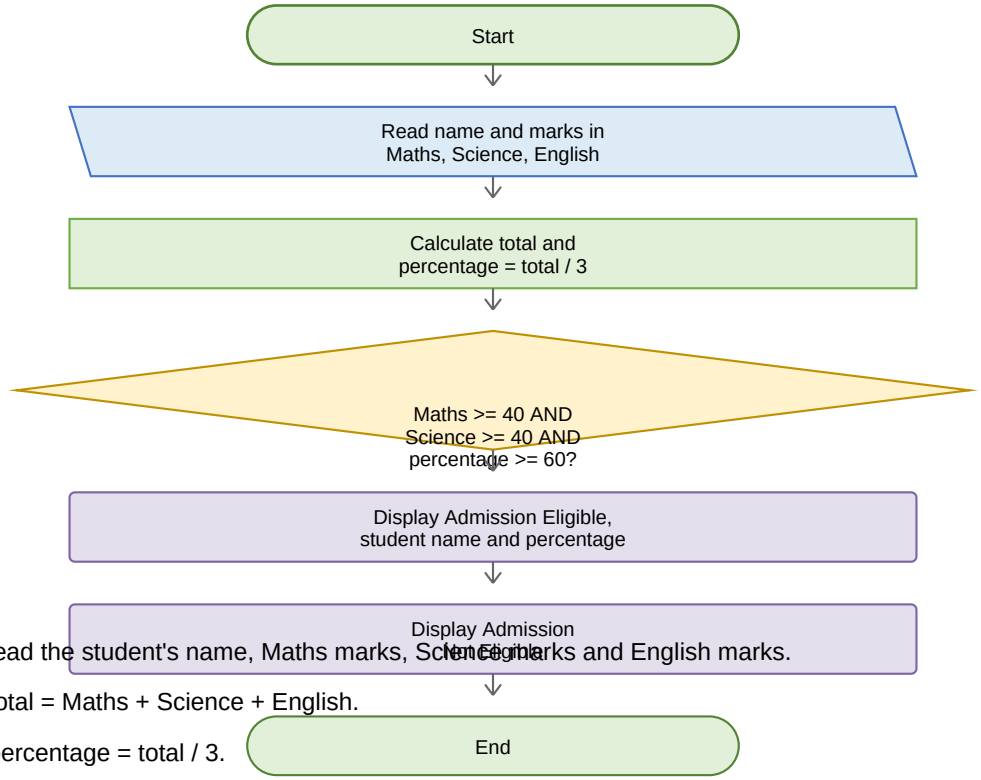
Task 6: Student Admission Eligibility

Aim

To determine admission eligibility using Maths marks, Science marks and the average percentage in three subjects.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start and read the student's name, Maths marks, Science marks and English marks.
2. Calculate total = Maths + Science + English.
3. Calculate percentage = total / 3.
4. Check whether Maths marks are at least 40, Science marks are at least 40, and percentage is at least 60.
5. If all conditions are true, display “Admission Eligible”, the student's name and percentage.
6. Otherwise, display “Admission Not Eligible”.
7. Stop the program.

Sample Input

```
Name: Anu
Maths: 70
Science: 65
English: 75
```

Expected Output

```
Admission Eligible
Student: Anu
Percentage: 70.0
```

Note

English marks contribute to the percentage but do not have a separate minimum-mark condition.

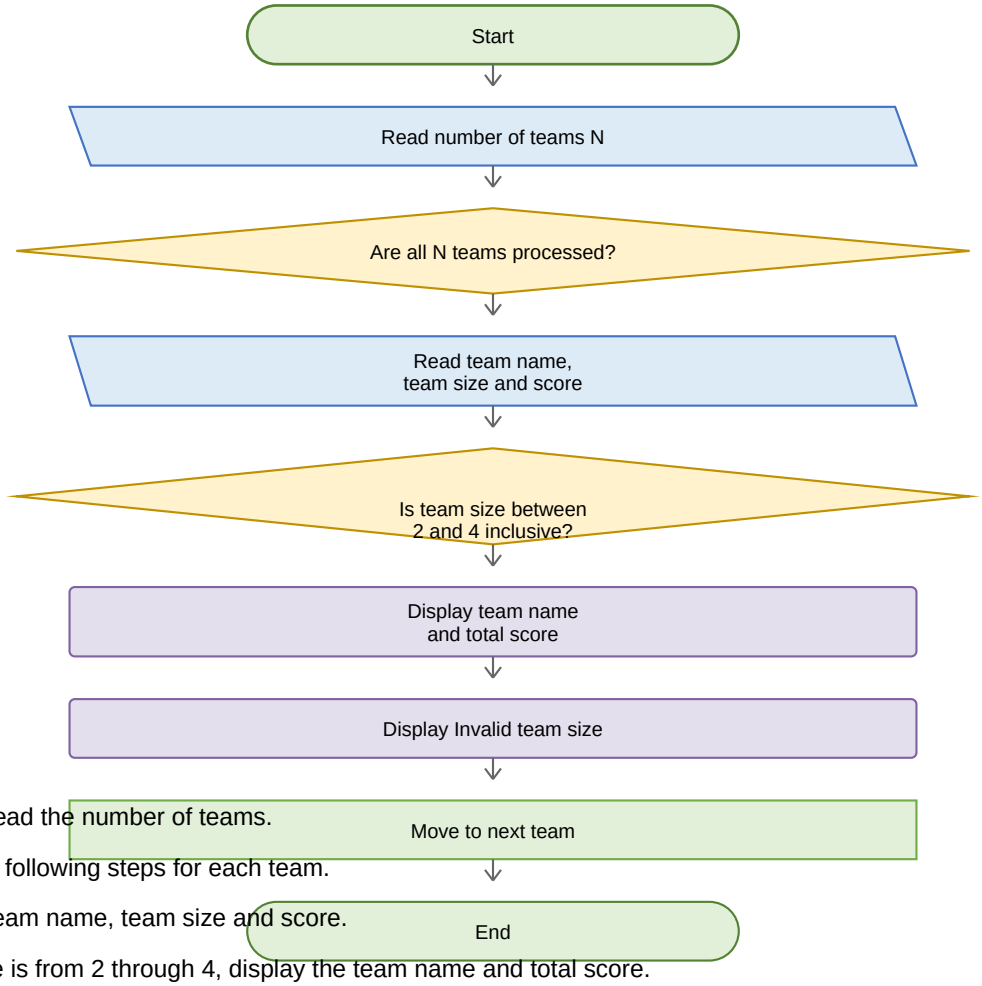
Task 7: Team Score Validation

Aim

To read details for multiple teams and display the score only when a team's size is between 2 and 4 inclusive.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start and read the number of teams.
2. Repeat the following steps for each team.
3. Read the team name, team size and score.
4. If team size is from 2 through 4, display the team name and total score.
5. Otherwise, display “Invalid team size”.
6. Continue until details for all teams have been processed, then stop.

Sample Input

```
Number of teams: 2
Team 1: Tigers, size 3, score 85
Team 2: Eagles, size 5, score 90
```

Expected Output

```
Tigers Total Score: 85
Invalid team size
```

Note

The program uses a for-loop, so the team input steps are repeated exactly N times.

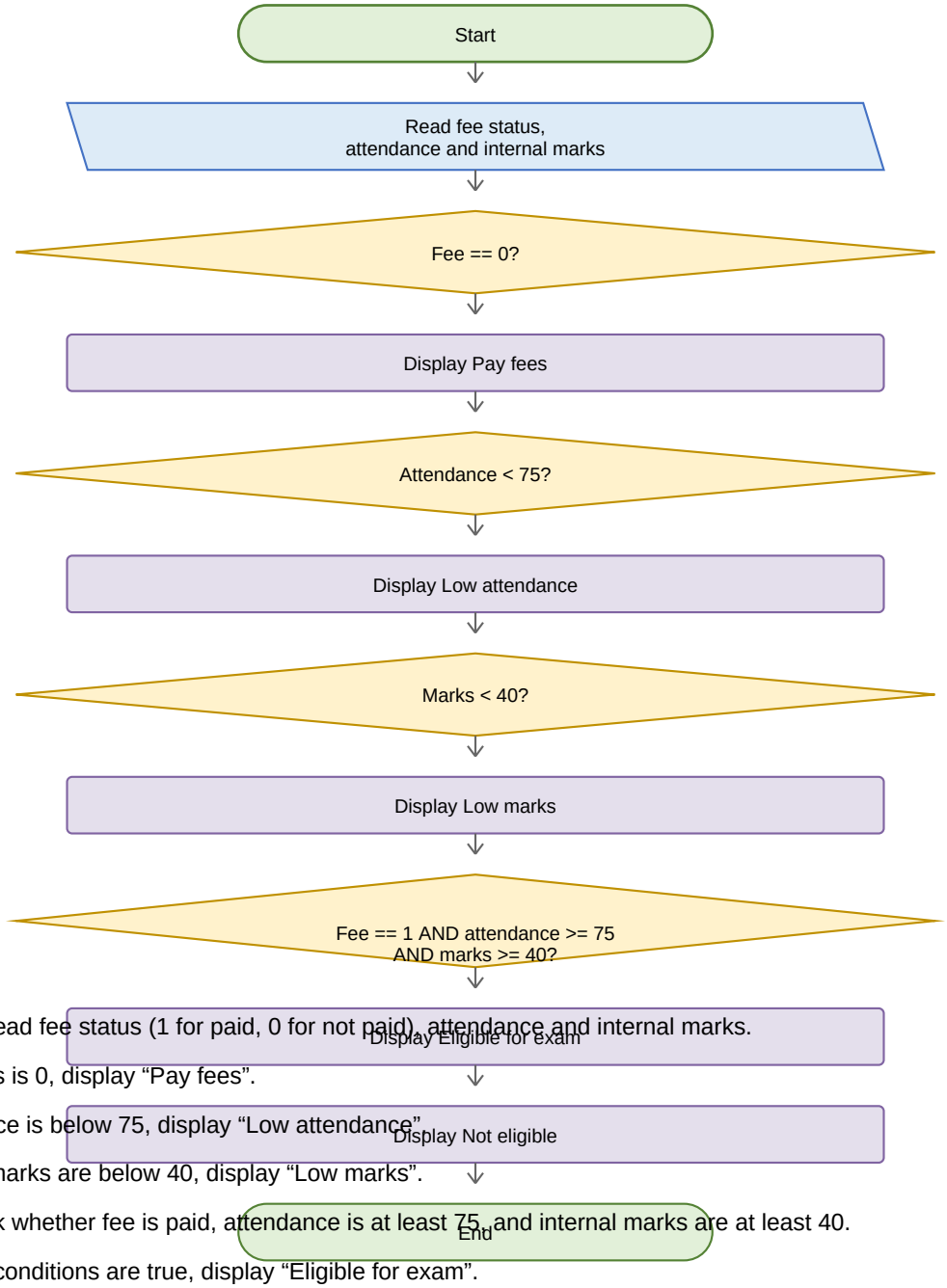
Task 8: Exam Eligibility Checker

Aim

To check exam eligibility using fee payment, attendance and internal marks, while displaying each applicable warning.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start and read fee status (1 for paid, 0 for not paid), attendance and internal marks.
2. If fee status is 0, display “Pay fees”.
3. If attendance is below 75, display “Low attendance”.
4. If internal marks are below 40, display “Low marks”.
5. Then check whether fee is paid, attendance is at least 75, and internal marks are at least 40.
6. If all three conditions are true, display “Eligible for exam”.
7. Otherwise, display “Not eligible”.
8. Stop the program.

Sample Input

```
Fee paid?: 1
Attendance: 80
Internal marks: 45
```

Expected Output

```
Eligible for exam
```

Note

The three warning checks are independent. More than one warning can be displayed before the final eligibility result.

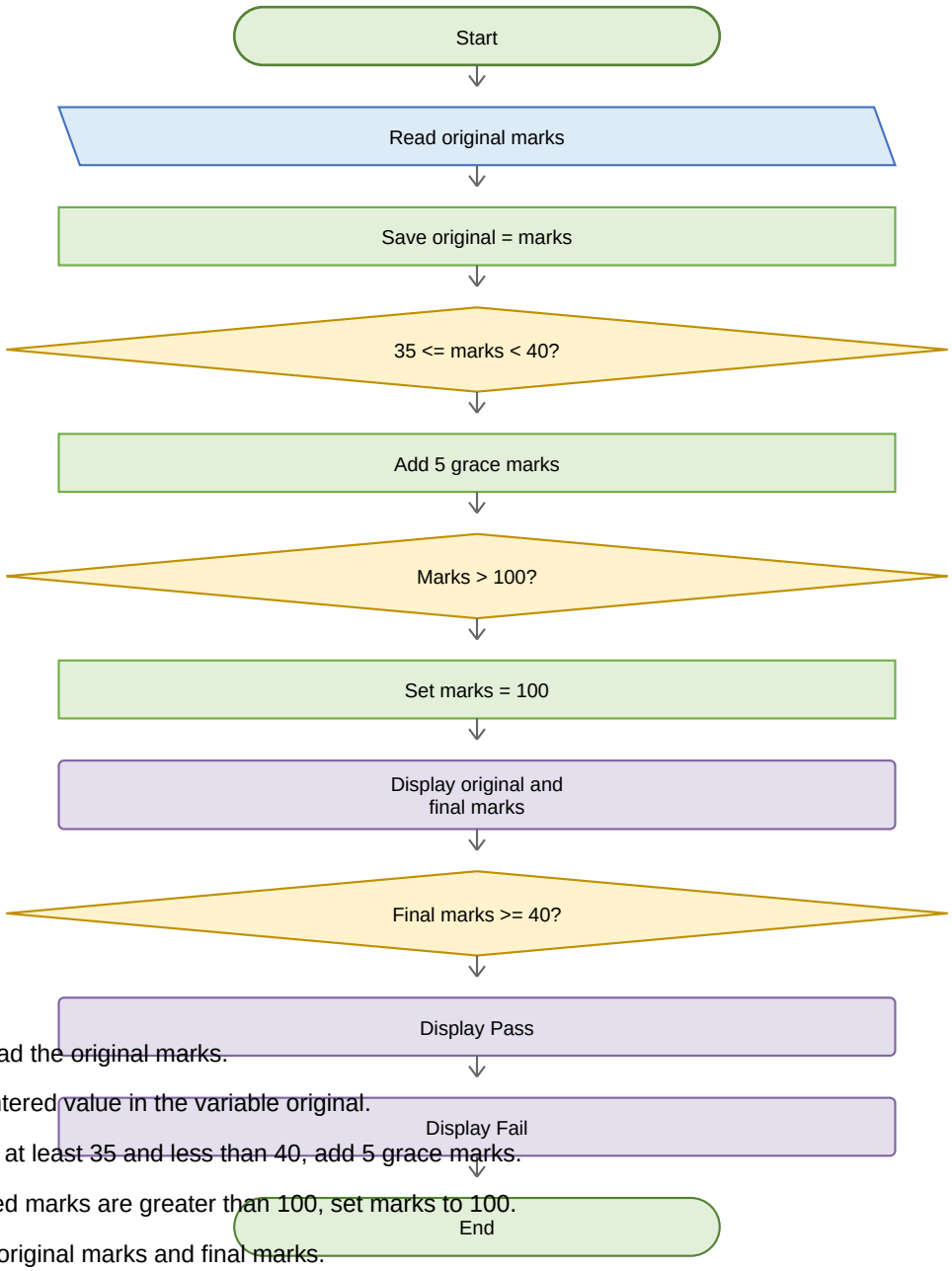
Task 9: Grace Marks and Pass/Fail

Aim

To adjust marks by adding five grace marks within the specified range, cap the final mark at 100, and display PASS or FAIL.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start and read the original marks.
2. Store the entered value in the variable original.
3. If marks are at least 35 and less than 40, add 5 grace marks.
4. If the updated marks are greater than 100, set marks to 100.
5. Display the original marks and final marks.
6. If final marks are at least 40, display PASS; otherwise, display FAIL.
7. Stop the program.

Sample Input

```
Original marks: 38
```

Expected Output

```
Original Marks: 38
Final Marks: 43
Pass
```

Note

Grace marks are added only when the original/current marks are from 35 to 39. The upper limit is applied after the grace-mark check.

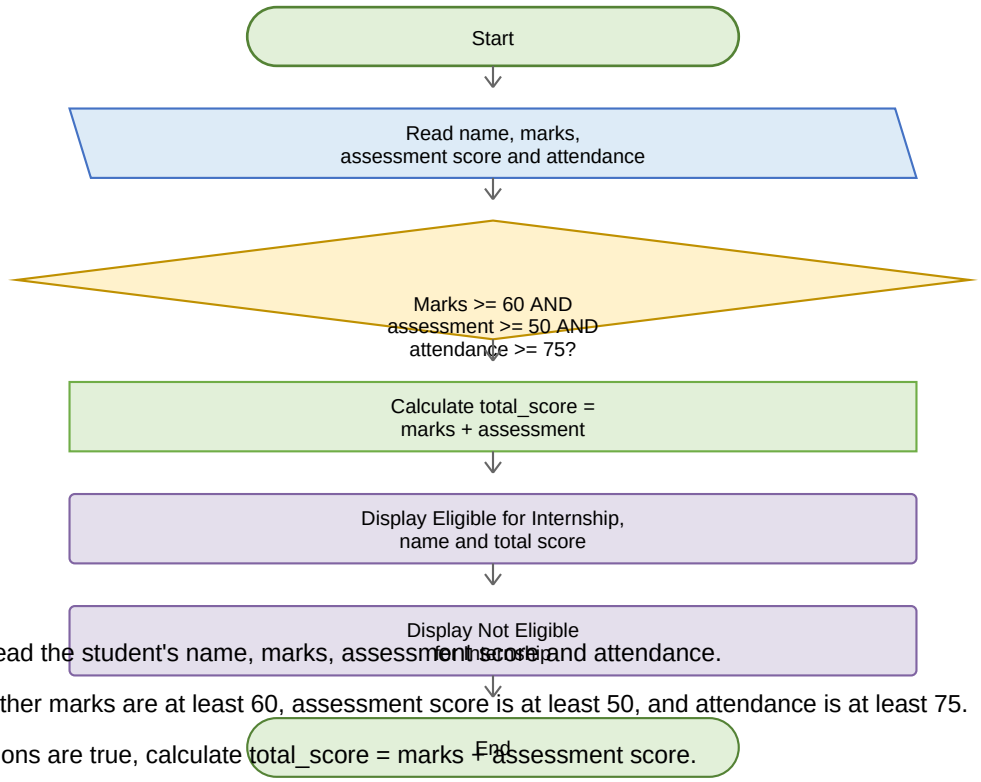
Task 10: Internship Eligibility Checker

Aim

To determine internship eligibility based on marks, assessment score and attendance, and calculate the combined score for eligible students.

Flowchart

Follow the arrows from top to bottom. Decision diamonds show the condition checked by the program.



Procedure

1. Start and read the student's name, marks, assessment score and attendance.
2. Check whether marks are at least 60, assessment score is at least 50, and attendance is at least 75.
3. If all conditions are true, calculate total_score = marks + assessment score.
4. Display “Eligible for Internship”, the student's name and total score.
5. Otherwise, display “Not Eligible for Internship”.
6. Stop the program.

Sample Input

```
Name: Kiran
Marks: 70
Assessment score: 60
Attendance: 80
```

Expected Output

```
Eligible for Internship
Student Name: Kiran
Total Score: 130
```

Note

The total score is calculated and displayed only when all three eligibility conditions are satisfied.